UNIVERSIDAD CATÓLICA ANDRÉS BELLO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA
CÁTEDRA DE SISTEMAS OPERATIVOS

INFORME TÉCNICO DE ARQUITECTURA Y MODELO DE CONCURRENCIA

Profesor:	Marcel J. Castro G.
Asignatura:	Sistemas Operativos (Semestre 2026-1)
Estándar y Lenguaje:	Lenguaje C (Estándar C11) / Hilos POSIX (pthreads)
Entorno Operativo:	POSIX Compliant / Windows (WinLibs UCRT MinGW-w64)
Entregable:	Informe de Arquitectura, Concurrencia y Declaración Ética de IA

1. Resumen Ejecutivo y Arquitectura del Sistema

El presente proyecto implementa un núcleo ejecutor (Kernel Simulator) y una Shell interactiva (CLI) para la gestión concurrente de procesos en sistemas robóticos embebidos de bajo consumo. La solución separa estrictamente la Entrada/Salida (Hilo CLI) de la Lógica de Ejecución (Hilo Motor de CPU), evitando que la interacción del usuario bloquee la simulación del tiempo discreto.

2. Modelo de Concurrencia y Sincronización (pthreads & mutex)

La arquitectura multihilo se implementa mediante la librería POSIX Threads (pthread). Ambos hilos comparten las estructuras del kernel y se sincronizan mediante un cerrojo de exclusión mutua (`pthread_mutex_t sistema_mutex`):

Hilo CLI (Consola)	Lectura interactiva no bloqueante mediante fgets, validación estricta de sintaxis/tipos sin colapsos, visualización de tablas ps, consulta de pcb y diagrama de gantt.	Bloquea sistema_mutex únicamente durante la consulta o inserción de nodos en las colas compartidas.
Hilo CPU (Scheduler)	Simulación de reloj discreto (+1 tick), admisión por lotes según Arrival Time, alternancia por Quantum (RR), desapropiación inmediata (SRTF), y cálculo de métricas de rendimiento.	Bloquea sistema_mutex al iniciar cada tick y lo libera antes de invocar el retardo real (Sleep/usleep) para no congelar la CLI.

```
DIAGRAMA DE SINCRONIZACIÓN Y ZONA CRÍTICA:

[Hilo CLI] -> pthread_mutex_lock() -> [COLAS COMPARTIDAS] <-
                pthread_mutex_lock() <- [Hilo CPU]
[Hilo CLI] <- pthread_mutex_unlock() <- (Ready, Running, Terminated) ->
                pthread_mutex_unlock() -> [Hilo CPU]
```

3. Estructura Modular del Código Fuente

kernel.h	Definición del PCB con métricas completas, estados del ciclo de vida (Estado Proceso), enum de algoritmos y variables globales externas.
kernel_core.c	Gestión de memoria dinámica, colas enlazadas (Ready, Running, Terminated, Batch), y parser robusto de carga por lotes (load).
cpu_scheduler.c	Motor de CPU simulado, reloj discreto, algoritmos Round Robin (RR) y SRTF apropiativo, desapropiación y registro de Gantt.
Interfaz_CL.c	Hilo CLI interactivo, validaciones léxicas/sintácticas con mensajes descriptivos sin colapso y formateo idéntico al estándar de evaluación.
Principal.c	Punto de entrada (main), inicialización, creación concurrente de hilos (pthread_create) y terminación limpia.
Makefile	Automatización de compilación con banderas estrictas (-Wall -Wextra -std=c11 -O2) y regla de limpieza (clean).

4. Algoritmos de Planificación y Modelado del PCB

Modelado del PCB: Cada proceso cuenta con un registro exhaustivo que rastrea su tiempo de llegada (Arrival Time, AT), primer acceso a CPU (Start Time, ST), tiempo de finalización (Completion Time, CT), ráfaga inicial (BT) y restante. A partir de estos valores se calculan matemáticamente las métricas del sistema:

- Tiempo de Retorno (Turnaround Time): $TAT = CT - AT$
- Tiempo de Espera (Waiting Time): $WT = TAT - BT$
- Tiempo de Respuesta (Response Time): $RT = ST - AT$
- Uso de CPU (%): (Ticks de CPU ocupada / Reloj total acumulado) × 100

Round Robin (RR): Asigna a cada proceso en estado READY un intervalo temporal fijo (Quantum, defecto 2 ticks). Si la ráfaga no culmina, el Hilo CPU desapropia el proceso, lo coloca al final de la cola y cede la CPU al siguiente nodo FIFO.

Shortest Remaining Time First (SRTF): Variante apropiativa de SJF. En cada tick, el motor evalúa si un nuevo proceso tiene menor tiempo restante que el que está en ejecución. De ser así, se desapropia de inmediato y se entrega la CPU al proceso más corto.

5. Resultados de Validación y Escenarios de Prueba

Escenario 1 (Manual e Interactivo): Se crearon P1 (BT 5 prio 2), P2 (BT 3 prio 1) y P3 (BT 2 prio 3) bajo RR con quantum 2. Se pausó en t=6 para inspección con pcb 1, se cambió en caliente a SRTF y se reanudó. Traza obtenida en Gantt:

`[0-2s: P1] → [2-4s: P2] → [4-6s: P3] → [6-7s: P2] → [7-10s: P1] → [IDLE]`

Escenario 2 (Carga en Lote con workload_test.txt): Se procesó el archivo bajo SRTF (Job_A: BT=4 AT=0; Job_B: BT=2 AT=1; Job_C: BT=1 AT=3). El motor detectó en t=1 la llegada de Job_B (BT=2), desapropiando de inmediato a Job_A (BT restante=3). La traza en Gantt coincidió con exactitud matemática con el valor esperado por la cátedra:

`[0-1s: Job_A] → [1-3s: Job_B] → [3-4s: Job_C] → [4-7s: Job_A] → [IDLE]`

6. Anexo Obligatorio: Declaración del Uso Ético de Inteligencia Artificial

En cumplimiento de las Secciones 8 y 9 del enunciado, se presenta la declaración formal del uso de IA:

Herramienta Utilizada	Antigravity AI (Google DeepMind) en entorno local.
Rol Asignado a la IA	Tutor socrático y asistente de depuración técnica. Se utilizó para identificar incompatibilidades de toolchains en Windows (diagnóstico de MinGW 2016 sin soporte de pthread.h e instalación guiada de WinLibs UCRT), resolución de conflictos léxicos y diseño concurrente.
Prompts Principales	• «Dime primero qué tengo que tener descargado en mi laptop para poder ejecutar el código...». • «Genérame un resumen de lo que realizó mi compañero para poder entenderlo...».
Validación Crítica	• Se diagnosticó que la macro SCHED_RR colisionaba con cabeceras de sistema POSIX, renombrándose a PLANIF_RR. • Se validaron manualmente los cálculos de tiempos de espera (WT), retorno (TAT) y respuesta (RT) para confirmar su rigor matemático.

7. Referencias Bibliográficas

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons. Capítulos 3, 4 y 5.
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson. Capítulo 2: Processes and Threads.
3. IEEE & The Open Group. (2018). POSIX.1-2017: Standard for Information Technology—Portable Operating System Interface (POSIX). IEEE Std 1003.1.
4. Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press. Capítulos 29-30.